Because each character is a collection of dots in a 5 × 8 grid, we can create an array of bytes, one byte for each row and one bit for each column. A 1 represents a pixel that is on and 0 a pixel that is off. Our example is written to save space, but it is common to write these arrays in the following format:

```
byte smiley[] = {
  B00000,
  B00000,
  B01010,
  B00000,
  B10001,
  B01110,
  B00000,
  B00000
};
```

While this way takes up more space on the screen, if you squint just right you can make out the smiley face in the code. With the array defined and the `custChar()` function called to assign our character to a position, we then use the `write()` function to display the character on the screen, like so:

```
lcd.write(0);
```

One useful utility for creating custom characters is the Character Creator shown in Figure 9-4 and found online at `http://icontexto.com/charactercreator/`.

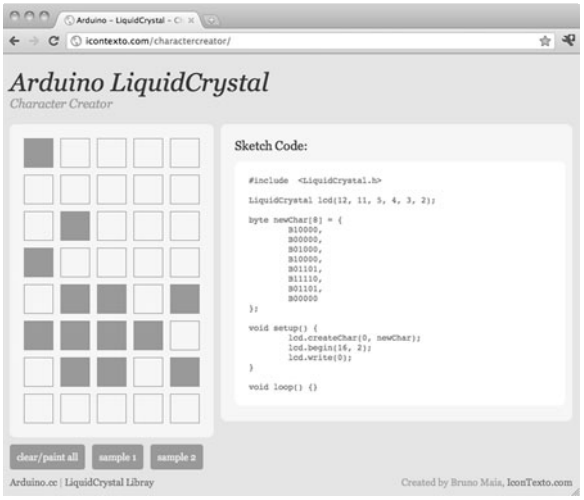


Figure 9-4. *Arduino LiquidCrystal Character Creator*

Simply toggle each block to draw your character and then copy and paste the generated code in the right panel into your Arduino sketch. And that's all there is to creating custom characters. The library fortunately takes on all the hard work of writing this information to the LCD's memory so that we can display this character, or even make a new character, whenever we want.

So now going back to our earlier example code that displays eight custom characters on the second row of our display, we first defined eight character arrays that included characters called `smiley`, `skull`,